КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

08

Лекция 8
Перегрузка операторов

Правила перегрузки · метод vs свободная функция · арифметика и составные операторы · ++/-- · сравнение и operator<=> · ввод/вывод · индексирование · функторы · operator-> · преобразования типа · пользовательские литералы

C++20 Операторы

ПЛАН ЛЕКЦИИ

Десять глав сегодняшней лекции

01 Правила перегрузки операторов

02 Арифметика и составные операторы

03 Префиксный и постфиксный ++/--

04 Сравнение и operator<=>

05 Операторы ввода/вывода

> **Сквозная идея:** оператор — это функция с синтаксисом выражения: перегрузка, конверсии и const-правильность работают здесь так же, как для обычных функций.

06 Оператор индексирования []

07 Оператор вызова () — функторы

08 Оператор -> — proxy-объекты

09 Преобразования типа

10 Пользовательские литералы и канонические формы

Перегрузка внутри и снаружи класса

```cpp
// Класс рационального числа (дробь).
class Fraction {
    int num_;  // числитель
    int den_;  // знаменатель
public:
    Fraction(int n = 0, int d = 1) : num_(n), den_(d) {}

    // --- Оператор как МЕТОД: левый операнд всегда Fraction ---
    // Подходит, когда левый операнд по смыслу не нуждается в преобразовании.
    Fraction operator*(const Fraction& rhs) const {
        return Fraction(num_ * rhs.num_, den_ * rhs.den_);
    }

    // --- Оператор как СВОБОДНАЯ функция (friend) ---
    // Нужен, чтобы работало "2 * frac": левый операнд — int, который
    // неявно преобразуется в Fraction. Метод здесь НЕ помог бы:
    // у int нет метода operator* для Fraction.
    friend Fraction operator*(int lhs, const Fraction& rhs) {
        return Fraction(lhs * rhs.num_, rhs.den_);
    }

    friend std::ostream& operator<<(std::ostream& os, const Fraction& f) {
        return os << f.num_ << '/' << f.den_;
    }
};
```

Пример
01_member_vs_free



Бинарный оператор — через составной

```
#include "vector2d.h"

int main() {
    Vector2D a(1.0, 2.0);
    Vector2D b(3.0, 4.0);

    std::cout << "a       = " << a << '\n';
    std::cout << "b       = " << b << '\n';
    std::cout << "a + b   = " << a + b << '\n';
    std::cout << "b - a   = " << b - a << '\n';
    std::cout << "-a      = " << -a << '\n';
    std::cout << "a * 2   = " << a * 2.0 << '\n';
    std::cout << "3 * b   = " << 3.0 * b << '\n';

    // Составной оператор изменяет объект на месте.
    a += b;
    std::cout << "после a += b: a = " << a << '\n';

    return 0;
}
```

Канон: реализовать +=, затем +через него — логика не дублируется.

Пример
02_arithmetic_compound



Постфиксный — int-заглушка и копия

```
16      // --- Постфиксный c++: int-заглушка отличает его от префиксного ---
17      // Возвращает КОПИЮ объекта ДО изменения.
18      Counter operator++(int) {
19          Counter old = *this;
20          ++(*this);   // переиспользуем префиксный
21          return old;
22      }
23
24      // --- Аналогично для -- ---
25      Counter& operator--() {
26          --value_;
27          return *this;
28      }
```

Пример
03_increment_decrement



Одна строка вместо шести операторов

Характеристика	`std::strong_ordering`	`std::weak_ordering`	`std::partial_ordering`
Полнота порядка	Полный (все значения сравнимы)	Полный (все значения сравнимы)	**Частичный** (возможны несравнимые значения)
Заменяемость	**Да** (`equal` ⇒ идентичность)	Нет (`equivalent` ⇒ возможны различия)	Нет (`equivalent` ⇒ возможны различия)
Именованные значения	`less` , `equal` , `greater`	`less` , `equivalent` , `greater`	`less` , `equivalent` , `greater` , `unordered`
Пример	`int` , `std::string` (с учётом регистра)	Строки без учёта регистра	`double` (из-за `NaN`)

Пример
04_comparison_spaceship



Только свободные функции

```cpp
// Комплексное число. Операторы потока — ВСЕГДА свободные функции,
// так как левый операнд — это поток (std::ostream / std::istream).
class Complex {
    double re_;
    double im_;
public:
    Complex(double r = 0.0, double i = 0.0) : re_(r), im_(i) {}

    // Вывод: возвращаем ссылку на поток для цепочки cout << a << b.
    friend std::ostream& operator<<(std::ostream& os, const Complex& c) {
        os << c.re_;
        if (c.im_ >= 0) os << '+';
        os << c.im_ << 'i';
        return os;
    }

    // Ввод: формат "re im", например "3 -4".
    friend std::istream& operator>>(std::istream& is, Complex& c) {
        is >> c.re_ >> c.im_;
        return is;
    }
};
```

Пример
05_io_operators



Две версии: const и non-const

```
46      // non-const: разрешает запись.
47      Row operator[](std::size_t i) {
48          if (i >= rows_) throw std::out_of_range("row index");
49          return Row(data_.data() + i * cols_, cols_);
50      }
51
52      // const: только чтение (для const Matrix& и const-методов).
53      ConstRow operator[](std::size_t i) const {
54          if (i >= rows_) throw std::out_of_range("row index");
55          return ConstRow(data_.data() + i * cols_, cols_);
56      }
```

Пример
06_subscript



Объект ведёт себя как функция

```
6   // Функтор: хранит состояние (offset_) между вызовами.
7   class Adder {
8       int offset_;
9   public:
10      explicit Adder(int offset) : offset_(offset) {}
11      int operator()(int x) const { return x + offset_; }
12  };
```

Пример
07_call_functor



Цепочка разыменований

```
22  // --- 1. Умный указатель: * возвращает ссылку, -> возвращает указатель ---
23  class WidgetPtr {
24      Widget* ptr_;
25  public:
26      explicit WidgetPtr(Widget* p = nullptr) : ptr_(p) {}
27      ~WidgetPtr() { delete ptr_; }
28
29      WidgetPtr(const WidgetPtr&) = delete;
30      WidgetPtr& operator=(const WidgetPtr&) = delete;
31
32      // *p — это сам объект (lvalue): можно читать и присваивать.
33      Widget& operator*() { return *ptr_; }
34      const Widget& operator*() const { return *ptr_; }
35
36      // p->member раскрывается в ptr_->member.
37      Widget* operator->() { return ptr_; }
38      const Widget* operator->() const { return ptr_; }
39  };
```

Пример
08_arrow_proxy



explicit спасает от ловушек

```
4   // Дробь с двумя операторами преобразования: неявным и явным.
5   class Fraction {
6       int num_;
7       int den_;
8   public:
9       Fraction(int n, int d = 1) : num_(n), den_(d) {}
10
11      // Неявное преобразование в double — естественно для дроби (значение).
12      operator double() const {
13          return static_cast<double>(num_) / den_;
14      }
15
16      // Явное (explicit) преобразование в bool — только в булевом контексте.
17      // Без explicit число случайно участвовало бы в арифметике через bool->int.
18      explicit operator bool() const { return num_ != 0; }
19  };
```

Пример
09_conversions



Синтаксис как API

```
 8
 9    // Пользовательские литералы делают код выразительным.
10    Money price = 100_rub;       // целочисленный литерал
11    Money discount = 49.90_rub;  // дробный литерал
12
13    std::cout << "цена    = " << price << '\n';
14    std::cout << "скидка  = " << discount << '\n';
15
```

Пример
10_literals_canonical



ИТОГИ

Ключевые концепции лекции

- **Метод vs свободная функция** — метод фиксирует левый операнд; свободная допускает конверсию.
- Канон +=-> +— бинарный оператор реализуется через составной.
- **Pre/Post increment** — префиксный возвращает ссылку, постфиксный — копию + int-заглушку.
- operator<=>(C++20) — одна строка вместо шести операторов.
- **Функторы** — объекты с operator(), совместимы с алгоритмами STL.
- explicit operator T()— запрет неявных опасных преобразований.
- **Пользовательские литералы** — выразительный синтаксис: 5.0_km.
- operator<</>>— только свободные функции, доступ через friend.